

Sistemas Operativos. Ejercicio de ficheros y sincronización. Julio de 2026

PROBLEMA [3 puntos]

Considere una partición con un tamaño de 4GiB en la que se crea un sistema de ficheros de tipo UNIX que usa agrupaciones de 4KiB, direcciones de agrupación de 32 bits, inodos de 128 bytes con 12 punteros directos, un indirecto simple, uno doble y uno triple, tal que está dimensionado suponiendo que el tamaño medio de un fichero es 1 agrupación.

Responda **razonadamente** a las siguientes cuestiones:

- [0,75 puntos]** Calcule qué porcentaje del tamaño de la partición ocupa la metainformación, obviando el sector de arranque y el *superbloque*, y no considerando la información contenida en los directorios como tal.
- [0,5 puntos]** Calcule qué tamaño tendrá un fichero que usa los punteros directos, la agrupación de indirección simple y las dos primeras entradas completas de la agrupación de indirección doble.
- [0,75 puntos]** El siguiente programa incompleto, que, para simplificar, no incluye control de errores, invierte byte a byte el contenido de un fichero (intercambia el primer y el último byte, el segundo y el penúltimo, etc.). Complete el programa, **manteniendo sin alterar las sentencias existentes**, especificando las nuevas sentencias y dónde se ubican estas con respecto a los puntos identificados.

```
int main(int argc, char *argv[]) {
    char c1, c2; struct stat st;
    int fd = open(argv[1], O_RDWR);
    fstat(fd, &st);
    for (size_t i=0; i < st.st_size/2; i++) {
        read(fd, &c1, 1); // PUNTO1: lee el siguiente carácter de la primera mitad del fichero
        read(fd, &c2, 1); // PUNTO2: lee el carácter correspondiente de la segunda mitad del fichero
        write(fd, &c1, 1); // PUNTO3: escribe el carácter de la primera mitad en el lugar correspondiente de la segunda
        write(fd, &c2, 1); // PUNTO4: escribe el carácter de la segunda mitad en el lugar correspondiente de la primera
    }
    close(fd);
    return 0;
}
```

- [1 punto]** Se requiere que cada acción que intercambia dos bytes se realice de forma **atómica**, pero permitiendo que, mientras dura esa acción, **otros procesos puedan acceder al resto del fichero** para leerlo o escribirlo.
 - Explique cuál es el mecanismo de sincronización más adecuado para solventarlo.
 - Especifique el código necesario indicando su ubicación con respecto a los puntos identificados y a las sentencias incluidas en el punto anterior.
 - Analice desde el punto de vista del interbloqueo la solución de sincronización planteada.

Solución

a) La metainformación corresponde a los mapas de bits de agrupaciones y de inodos, así como a la zona de inodos.

Cálculo de cuántas agrupaciones ocupa el mapa de bits de agrupaciones:

- n.º de agrupaciones en la partición = tamaño disco/tamaño agrupación = $4\text{GiB}/4\text{KiB} = 2^{32}/2^{12} = 2^{20}$ agrupaciones
- n.º bits en el mapa de bits de agrupaciones = 1 bit por cada agrupación en la partición = 2^{20} bits
- n.º bits caben en una agrupación = $4\text{KiB} * 8 = 2^{15}$ bits
- n.º agrupaciones mapa de bits agrupaciones = bits del mapa/bits por agrupación = $2^{20} / 2^{15} = 2^5$ **agrupaciones**

Cálculo de cuántas agrupaciones ocupa el mapa de bits de inodos; dado que el tamaño medio de un fichero es igual al de una agrupación, va a haber tantos inodos como agrupaciones y, por tanto, el tamaño de este mapa será igual que el del anterior:

- n.º de inodos en la partición = n.º de agrupaciones en la partición = 2^{20} inodos
- n.º agrupaciones mapa de bits inodos = n.º agrupaciones mapa de bits agrupaciones = 2^5 **agrupaciones**

Cálculo de cuántas agrupaciones ocupa la zona de inodos:

- n.º de bytes en la zona de inodos = n.º inodos * tamaño del inodo = $2^{20} * 128 = 2^{27}$ bytes
- n.º agrupaciones zona de inodos = bytes zona inodos/bytes por agrupación = $2^{27} / 2^{12} = 2^{15}$ **agrupaciones**

Por tanto, el porcentaje de ocupación de la partición por parte de la metainformación es:

- $(2^5 + 2^5 + 2^{15}) / 2^{20} = 3,13\%$

b) El tamaño del fichero se va a determinar calculando cuántos punteros usa, para lo que se requiere conocer cuántos punteros caben en una agrupación:

- n.º de punteros en una agrupación = tamaño agrupación/tamaño puntero = $4\text{KiB}/4\text{B} = 2^{12}/2^2 = 1024$ punteros

A continuación, se realiza el recuento de uso de punteros por parte del fichero tal como especifica el enunciado:

- Todos los punteros directos: 12.
- Todos los punteros de la agrupación de indirección simple: 1024.
- Todos los punteros de la agrupación apuntada por la 1ª entrada de la agrupación de indirección doble: 1024.
- Todos los punteros de la agrupación apuntada por la 2ª entrada de la agrupación de indirección doble: 1024.

Por tanto, el fichero ocupa 3084 ($12 + 3 * 1024$) agrupaciones.

En consecuencia, dependiendo del estado de ocupación de la última agrupación, que puede contener un solo byte o estar completamente usada, el fichero puede tener un tamaño entre:

- un mínimo de $3083 * 4096 + 1 = 12\,627\,969$ bytes
- un máximo de $3084 * 4096 = 12\,632\,064$ bytes.

c) En el código especificado hay que añadir las llamadas al servicio *lseek* necesarias para su buen funcionamiento. Hay que tener en cuenta que hay múltiples soluciones a esta cuestión dependiendo de cómo se use este servicio. A continuación, se muestra una posible solución.

```

int main(int argc, char *argv[]) {
    char c1, c2; struct stat st;
    int fd = open(argv[1], O_RDWR);
    fstat(fd, &st);
    for (size_t i=0; i < st.st_size/2; i++) {
        read(fd, &c1, 1); // PUNTO1: lee el siguiente carácter de la primera mitad del fichero
        lseek(fd, -(i+1), SEEK_END);
        read(fd, &c2, 1); // PUNTO2: lee el carácter correspondiente de la segunda mitad del fichero
        lseek(fd, -1, SEEK_CUR);
        write(fd, &c1, 1); // PUNTO3: escribe el carácter de la primera mitad en el lugar correspondiente de la segunda
        lseek(fd, i, SEEK_SET);
    }
    close(fd);
    return 0;
}

```

- d) Se requiere poder realizar de forma atómica la acción de intercambiar dos bytes del fichero dejando acceso libre al resto mientras dura esa operación. Para ello, el mecanismo de cerrojos de ficheros es el más adecuado al permitir establecer el acceso exclusivo a rangos de bytes, que es justo lo que se necesita en este escenario. Para lograr la sincronización requerida, se necesitan dos cerrojos, uno por cada byte involucrado en una operación de intercambio, que son tomados de forma exclusiva al inicio de cada iteración del bucle, afectando a los bytes respectivos, y liberados al final de la iteración. A continuación, se muestra una posible solución.

```

int main(int argc, char *argv[]) {
    char c1, c2; struct stat st;
    int fd = open(argv[1], O_RDWR);
    fstat(fd, &st);
    struct flock fl1, fl2;
    for (size_t i=0; i < st.st_size/2; i++) {
        fl1 = (struct flock) {.l_type=F_WRLCK, .l_whence=SEEK_SET, .l_start=i, .l_len=1};
        fl2 = (struct flock) {.l_type=F_WRLCK, .l_whence=SEEK_END, .l_start=-(i+1), .l_len=1};
        fcntl(fd, F_SETLKW, &fl1);
        fcntl(fd, F_SETLKW, &fl2);
        read(fd, &c1, 1); // PUNTO1: lee el siguiente carácter de la primera mitad del fichero
        lseek(fd, -(i+1), SEEK_END);
        read(fd, &c2, 1); // PUNTO2: lee el carácter correspondiente de la segunda mitad del fichero
        lseek(fd, -1, SEEK_CUR);
        write(fd, &c1, 1); // PUNTO3: escribe el carácter de la primera mitad en el lugar correspondiente de la segunda
        lseek(fd, i, SEEK_SET);
        write(fd, &c2, 1); // PUNTO4: escribe el carácter de la segunda mitad en el lugar correspondiente de la primera
        fl1.l_type=F_UNLCK;
        fl2.l_type=F_UNLCK;
        fcntl(fd, F_SETLKW, &fl1);
        fcntl(fd, F_SETLKW, &fl2);
    }
    close(fd);
    return 0;
}

```

El uso simultáneo de dos cerrojos posibilitaría la aparición de interbloqueos siempre que otro proceso solicitara los recursos en el orden inverso. Por tanto, no puede aparecer un interbloqueo en la ejecución de varias instancias de este programa sobre el mismo fichero puesto que todas ellas solicitan los recursos en el mismo orden: primero el byte de la primera mitad y, a continuación, el byte de la segunda mitad.